

L4

模型微调与 私有化大模型

3. LORA微调原理与实例

深度学习基础知识

- 机器学习

- 从数据中学习规律

- 监督学习：

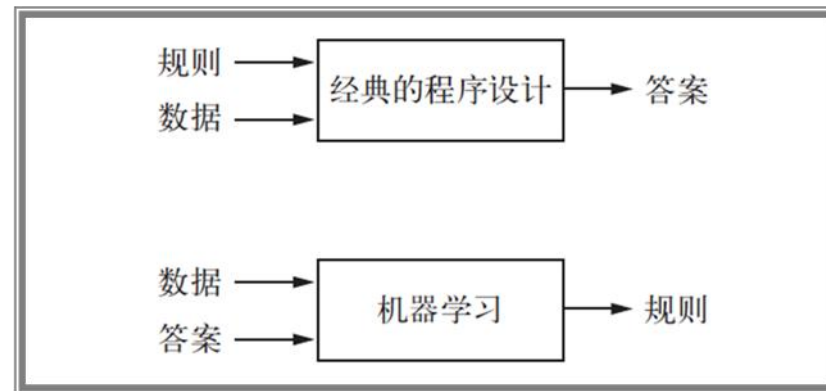
- $Y = f(X)$ ，有 $X \rightarrow Y$ 的大量数据，推算出 f 来

- 深度学习

- 利用深度神经网络进行机器学习
 - 深度神经网络（3层以上的神经网络）

- 深度学习的优势

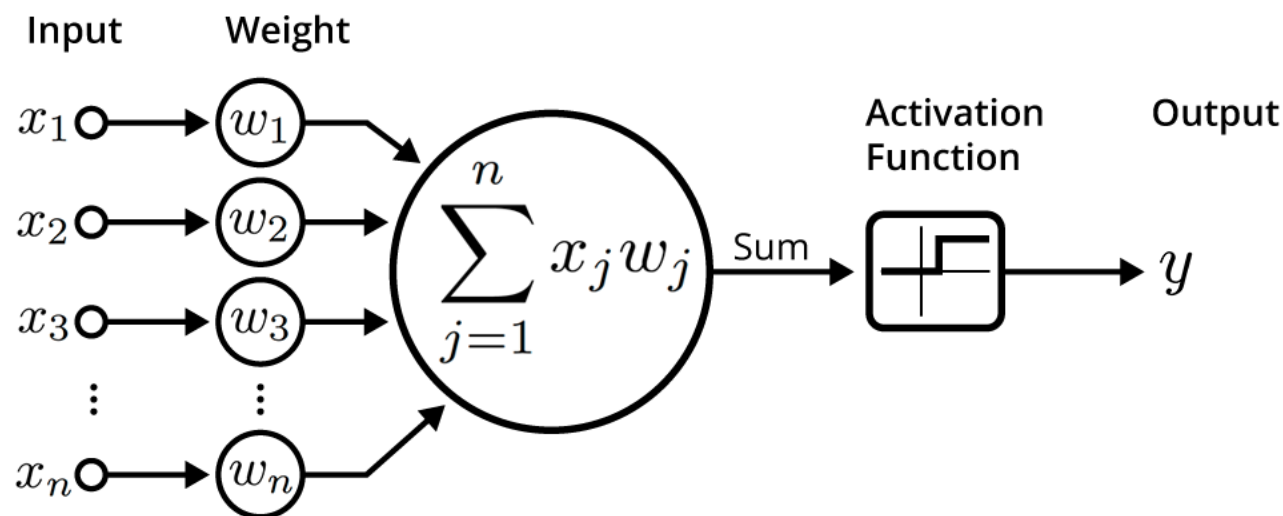
- 自动化：深度学习无需手工设计特征，效率高，效果往往优于人工特征。
 - 通用性：学习得到的特征适应性强，深度学习提供了灵活且通用的问题表示框架。
 - 能力强：可构建大型模型并充分利用海量数据。



- 神经元：最基本的计算单元

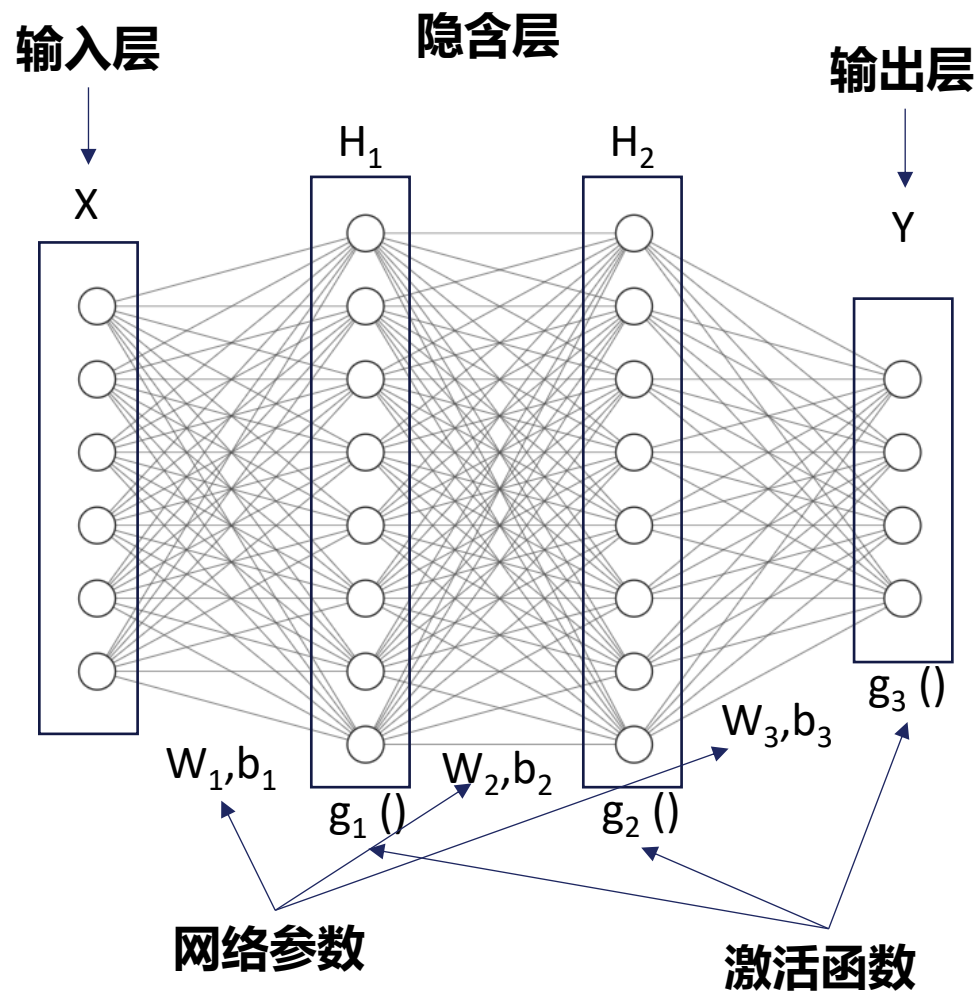
- 线性运算： $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$
- 非线性运算： $y = g(z)$

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$



An illustration of an artificial neuron. Source: Becoming Human.

w_i 和 b 叫做网络参数



整体表示

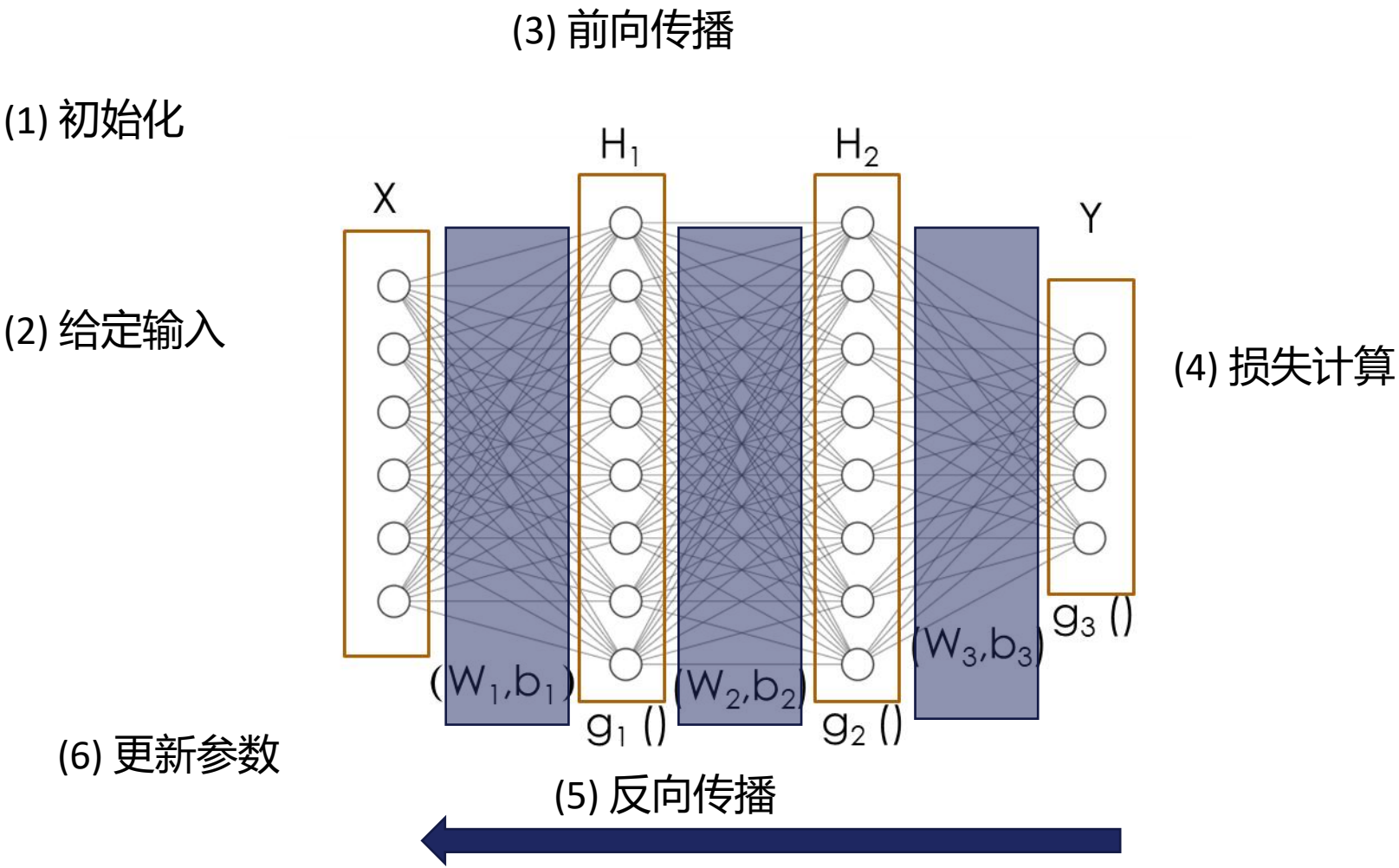
$$Y = f(X)$$

用矩阵表示计算过程

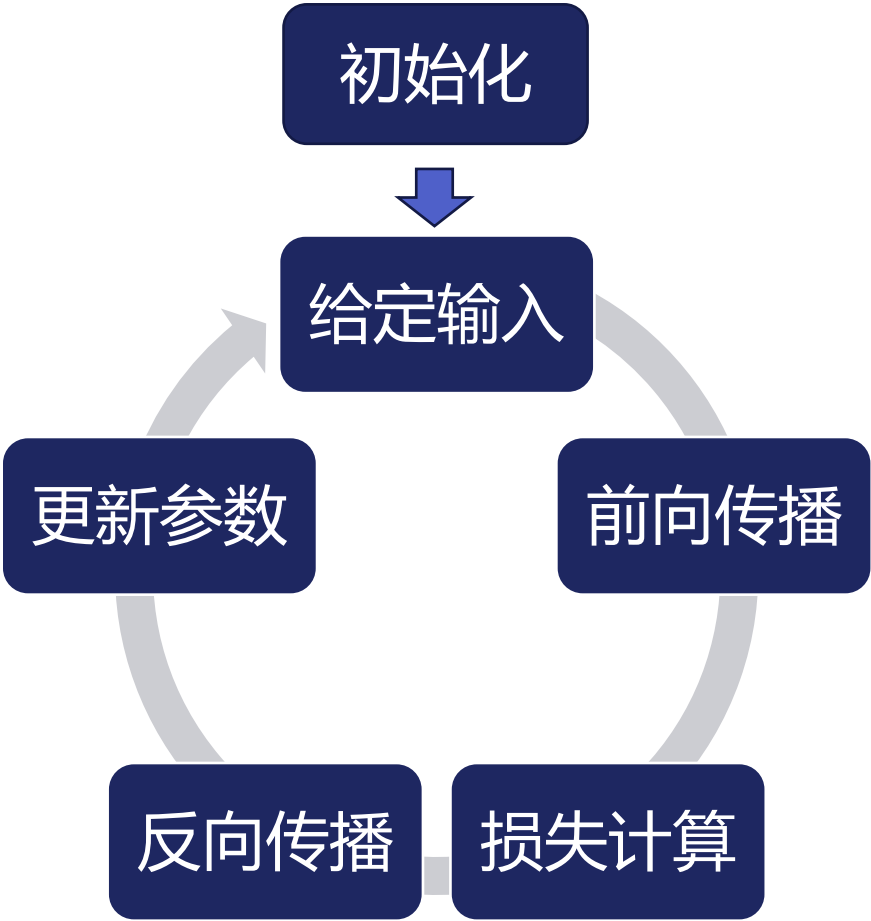
$$H_1 = g_1(W_1 X + b_1)$$

$$H_2 = g_2(W_2 H_1 + b_2)$$

$$Y = g_3(W_3 H_2 + b_3)$$



步骤	作用（简单说明）
初始化	给网络设置 初始参数 （随机设置）。
给定输入	给定一个 训练数据 样本。
前向传播	通过网络计算，得到 预测结果 。
损失计算	计算预测与 真实答案 的差距（误差）。
反向传播	把误差向前追溯， 找出参数应该怎么改 。
更新参数	调整网络参数，让 下次预测更准确 。



1. 准备阶段

- 加载模型并初始化参数
- 设置训练超参数（如学习率、批大小、训练轮数）
- 准备训练集和验证集

2. 多次循环训练验证

1. 训练阶段（Training）

- 逐批输入训练数据
- 计算损失并更新模型参数

2. 验证阶段（Validation）

- 使用验证数据评估模型
- 不更新参数，仅观察性能变化

3. 模型记录与保存

- 记录训练和验证结果
- 可保存表现最佳的模型

4. 完成与部署

- 训练结束后切换到推理模式
- 模型用于预测或部署到应用场景

```
开始训练
|
| 初始化模型参数
| 定义损失函数与优化器
| 准备训练集与验证集
| 设定训练轮数 (num_epochs)
|
| for epoch in range(num_epochs): ← 每一轮训练开始
| |
| | 1. 启动训练阶段 (Training Loop)
| |
| | 设置模型为训练模式: model.train()
| | 对训练集执行训练过程 (参数更新, 不展开具体步骤)
| | 统计训练阶段指标 (如训练 loss)
| |
| | 2. 启动验证阶段 (Validation Loop)
| |
| | 设置模型为评估模式: model.eval()
| | 对验证集执行评估过程 (不更新参数, 不展开具体步骤)
| | 统计验证阶段指标 (如验证 loss、accuracy 等)
| |
| | (可选) 判断是否满足早停条件 (Early Stopping)
| | (可选) 保存当前最佳模型 (Checkpoint)
| |
| | 返回 model.train(), 准备进入下一轮训练
|
└ 训练结束
    |
    | 使用测试集或实际数据进行推理 (Inference)
    └ 模型部署与应用
```

Epoch (训练轮次) 1 ~ 3 (LLM 微调)

- 一次 **完整遍历全部训练数据** 的过程称为一个 *epoch*。
- 模型会循环多轮训练，逐步学习和优化参数。
- 轮次过少 → 学习不足；轮次过多 → 可能过拟合。

Batch (批次) 1 ~ 4 (大模型) 16 ~ 128 (小模型)

- 数据不是一次全部送入模型，而是**按小批量 (batch) 分次训练**。
- batch 越大：训练更稳定，但越耗显存。

Loss Function (损失函数) CrossEntropyLoss

- 测量**模型预测与真实结果之间的差距**。
- 训练的目标是**让损失值越来越小**。

Optimizer (优化器) AdamW

- 根据梯度 **更新模型参数** 的算法，例如：SGD、Adam。
- 决定模型怎样“学习”。

Learning Rate (学习率) (1e-4 ~ 2e-5)

- 参数更新时迈出的步伐大小。
- 太大 → 训练不稳定；太小 → 收敛太慢。

大模型架构简介

Transformer

特点:

- 输入输出均为序列
- 编码器和解码器结构
- 自注意力机制

优势

- **长距离依赖**: 能直接关联序列任意位置, 捕捉长程依赖更容易。
- **并行计算**: 自注意力可并行处理整个序列, 训练更高效。
- **可扩展性**: 多头注意力和堆叠层数灵活, 易于扩展到大规模模型 (如 BERT、GPT 系列) 。

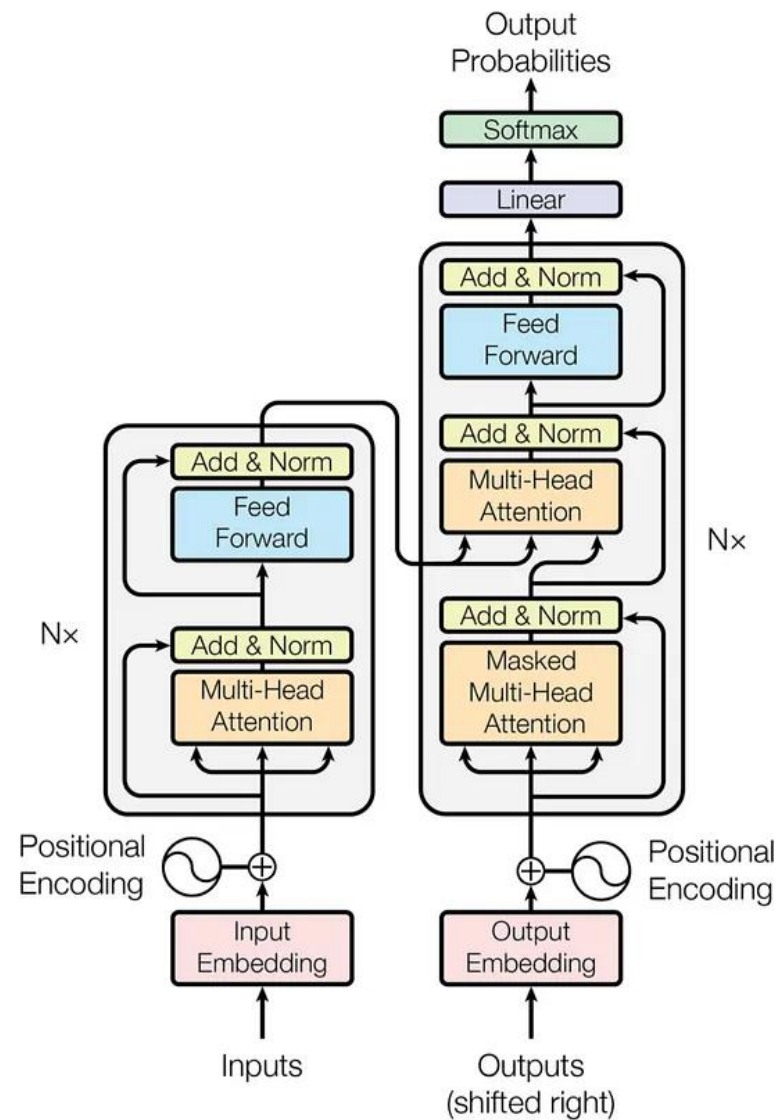


Figure 1: The Transformer - model architecture.

Transformer整体架构概览

核心设计

采用 Encoder-Decoder 框架，基于注意力机制构建。

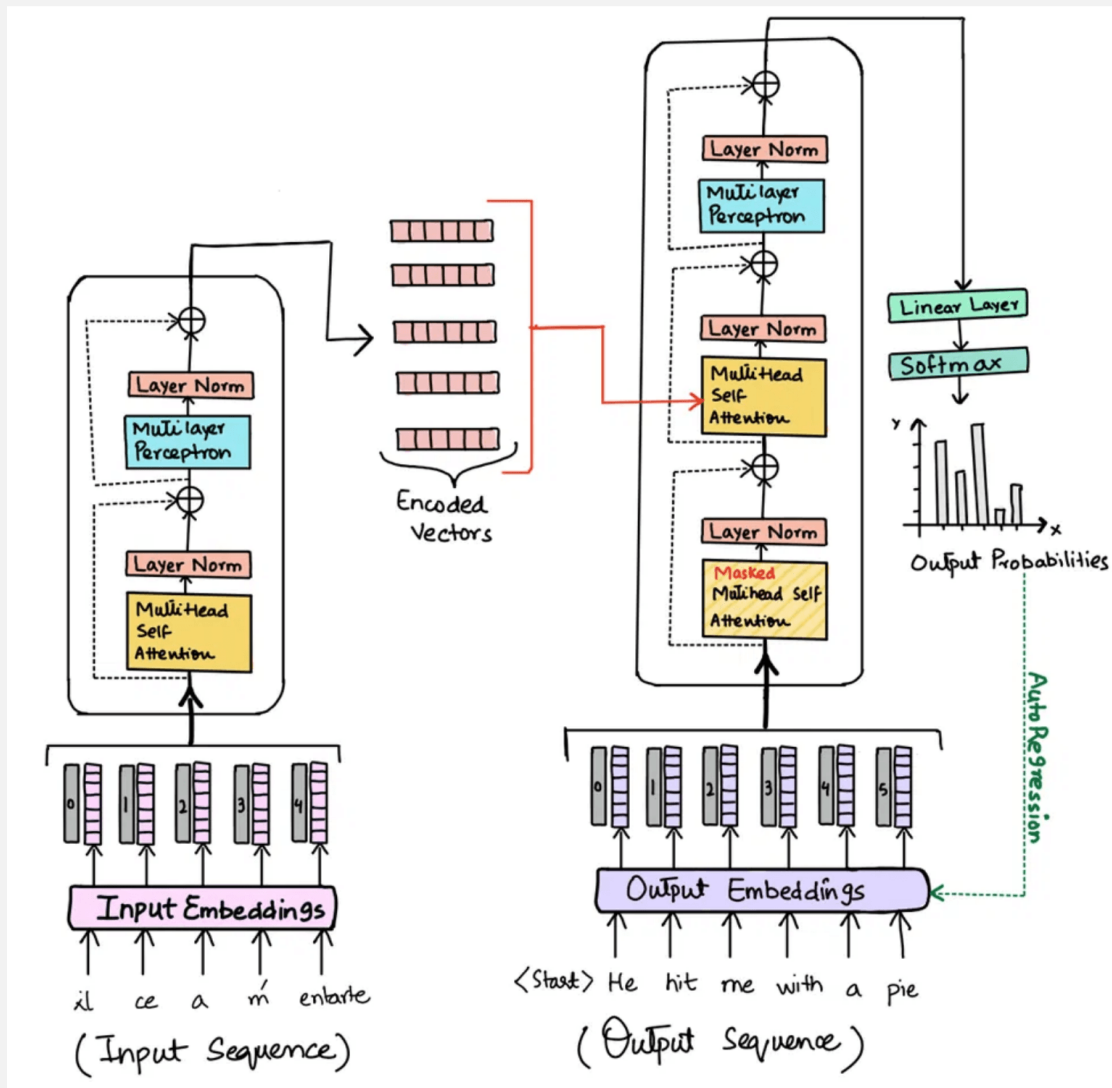
关键组件

Encoder (编码器): 将输入序列映射为高维语义表示，捕捉全局上下文。

Decoder (解码器): 基于编码器输出和已生成序列，逐步预测下一个Token。

数据流向

- 1 输入文本 → 词嵌入 (Embedding)
- 2 注入位置信息 (Positional Encoding)
- 3 Encoder 提取特征 → Decoder 生成分布
- 4 Softmax 输出概率 → 目标序列



编码器：理解输入文本

文本转向量：

- 输入文本拆分成“词元”（Token），每个 Token 转换为固定长度的数字向量（含语义和位置信息）。

自注意力机制：

- 每个词向量会“关注”所有词向量，计算彼此关联度，融合成包含全局信息的新向量（如“猫”会结合“坐”“垫子”的信息）。

多层处理：

- 上述过程重复多层，每层增强特征，最终输出“编码向量”，包含输入文本的整体语义。

解码器：生成目标文本

从起始符开始：

- 以特殊符号（如<开始>）作为初始输入，转换为向量。

逐词生成：

- 每次生成一个词，添加到已生成序列中，作为下一步输入（如先生成第一个词，再基于它生成第二个词）。

关联输入信息：

- 生成每个词时，会“查询”编码器的编码向量，建立输入文本与当前生成词的语义关联（如翻译时对齐源词和目标词）。

结束生成：

- 遇到特殊结束符（如<结束>）或达到预设长度时停止，最终将生成的词序列转换为自然语言文本。

自注意力完整流程（五步）

01

输入Embedding

每个token转换为d维向量；Qwen1.5-1.8B: $d=2048$ ；长度n的序列 $\rightarrow n \times 2048$ 矩阵

02

线性投影得到Q/K/V

输入矩阵分别乘以权重矩阵 W_q 、 W_k 、 W_v $\rightarrow$ 得到Query、Key、Value

★ 这三个矩阵就是LoRA最核心的注入点

03

计算注意力分数

Q和K做点积 $\rightarrow$ 除以 $\sqrt{d_k}$ 缩放 $\rightarrow$ softmax归一化为概率分布

注意力分数 = 每个token应关注其他token的程度

04

加权求和

用注意力分数对V矩阵加权求和 $\rightarrow$ 得到融合上下文信息的新表示

05

输出投影

加权求和结果乘以输出投影矩阵 W_o $\rightarrow$ 该层自注意力的最终输出

自注意力机制

• 公式：

Query (查询) : $Q = XW^Q$

Key (键) : $K = XW^K$

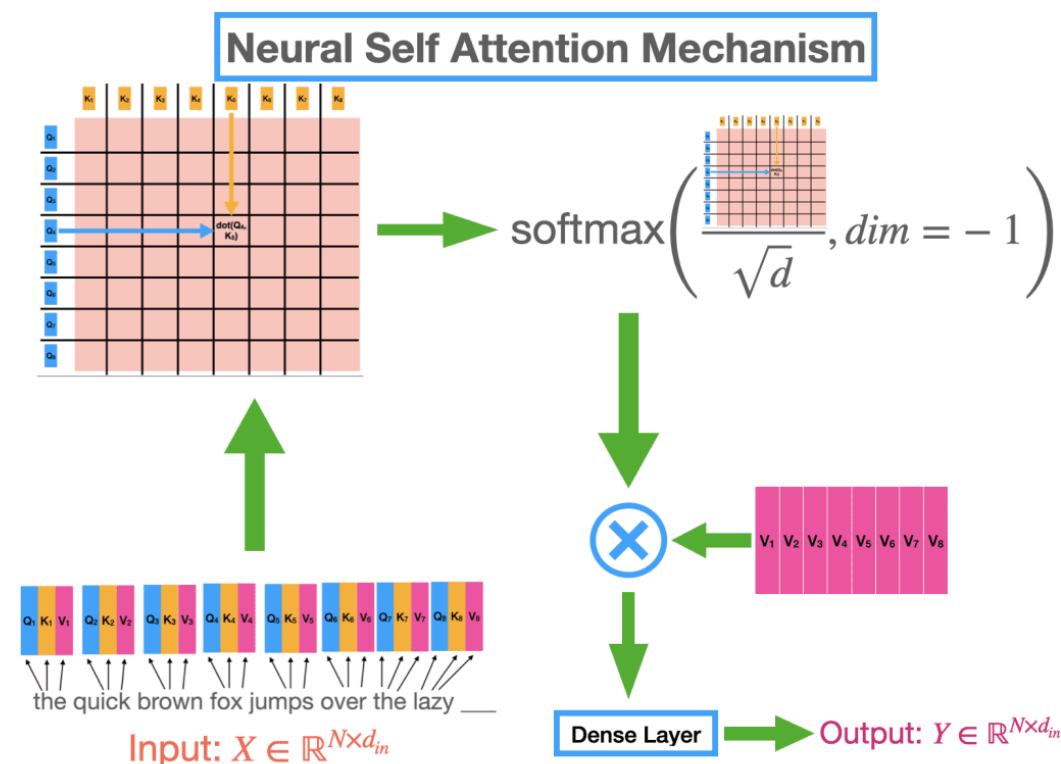
Value (值) : $V = XW^V$

Attention(Q, K, V) = $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

• 直观理解：

对于序列中某个位置的词，自动“关注”序列中所有位置（包括自己），并根据它们与这个词的重要性加权汇总信息。

• 示意图



核心机制：缩放点积注意力

核心公式： $\text{Attention}(Q, K, V) = \text{softmax}(\mathbf{QK}^T / \sqrt{d_k}) \cdot V$



Query (查询)

当前位置想要
询问什么信息
(我在找什么?)



Key (键)

每个位置对外
宣传自己的特征
(我能提供什么?)



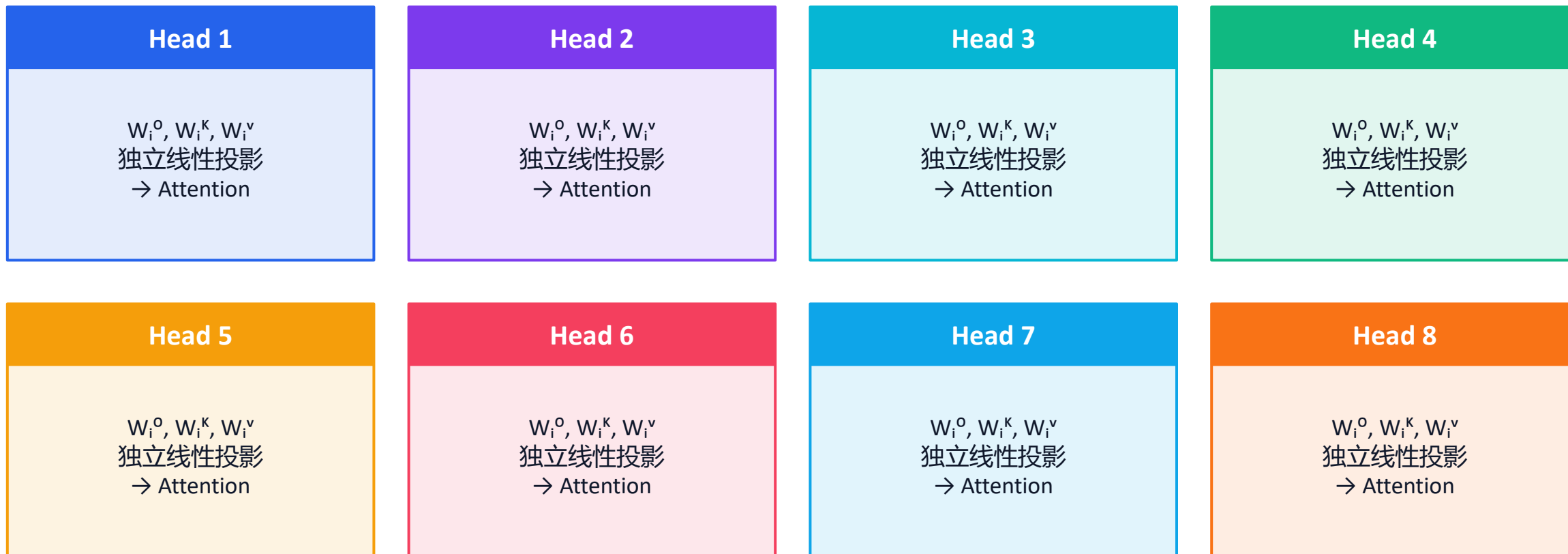
Value (值)

每个位置实际
携带的信息内容
(具体的信息是...)

计算流程： ① 矩阵乘法 $Q \cdot K^T$ → ② 除以 $\sqrt{d_k}$ 缩放 → ③ Softmax 归一化 → ④ 乘以 V 加权求和

多头注意力机制 (Multi-Head Attention)

用 h 个独立的注意力头，从不同子空间学习信息，再拼接融合——一次看多个角度



$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_8) \rightarrow \text{Linear}(W^O) \rightarrow \text{输出}$

优势：捕捉不同类型的语义关系（句法、语义、指代...）；8头并行计算，效率高

大模型微调方法

全参数微调 (Full Fine-tuning)



定义：更新模型的所有参数。示例：Qwen1.5-1.8B 约18亿参数，全部计算梯度并更新。

三大问题

1

显存占用巨大

模型本身 (FP16约3.6GB) + 优化器状态 (AdamW需存一阶和二阶动量, 约模型大小2~3倍) + 梯度 + 激活值
总显存可达模型大小的4~6倍, 7B以上模型基本需要多卡训练

2

灾难性遗忘

在小数据集上全量微调, 模型可能"忘掉"预训练阶段的通用能力, 导致在目标任务之外表现下降

3

存储成本高

每个下游任务需保存一份完整模型副本, 部署管理成本极高

PEFT 核心思想：冻结预训练模型大部分参数，只训练少量新增或选定的参数，既能适配下游任务，又能大幅降低计算和存储成本。

Adapter Tuning (2019)

Houlsby等人

每层自注意力和FFN后插入小型适配器：下投影($d \rightarrow r$) $\rightarrow$ 激活 $\rightarrow$ 上投影($r \rightarrow d$)。参数量1%~5%。
缺点：推理时串行计算，增加推理延迟

Prefix Tuning (2021)

Li和Liang

每层注意力K/V前拼接可学习"前缀"向量，相当于给每层提供额外引导信号。
缺点：占用上下文长度（前缀挤占输入空间），效果不如LoRA稳定

Prompt Tuning (2021)

Lester等人

只在输入embedding层前拼接可学习"软提示"向量，其余层不改，参数量极小。
缺点：中小模型效果有限，通常需10B+模型才能接近全参微调效果

LoRA (2021) ★

Hu等人

原始权重矩阵旁并行添加低秩分解 $\Delta W = BA$ 。前向：输出 = $Wx + BAx$ 。参数量仅1%~2%。
三大优势：效果好、推理无额外延迟（可合并）、权重可拆卸（灵活部署）

1

效果好

多数任务上接近甚至媲美全参数微调, 明显优于 Prompt Tuning / Prefix Tuning

2

推理无额外延迟

训练后可将 BA 合并到 W 中 ($W'=W+BA$), 推理时结构不变、无额外开销
而 Adapter 在推理时仍需经过额外层, 增加延迟

3

权重可合并与可拆卸

切换任务只需更换一组小的 LoRA 权重文件 (几十~几百 MB)
不需保存完整模型, 部署灵活, 多任务支持成本极低

→ LoRA 成为当前大模型微调的事实标准

核心思想

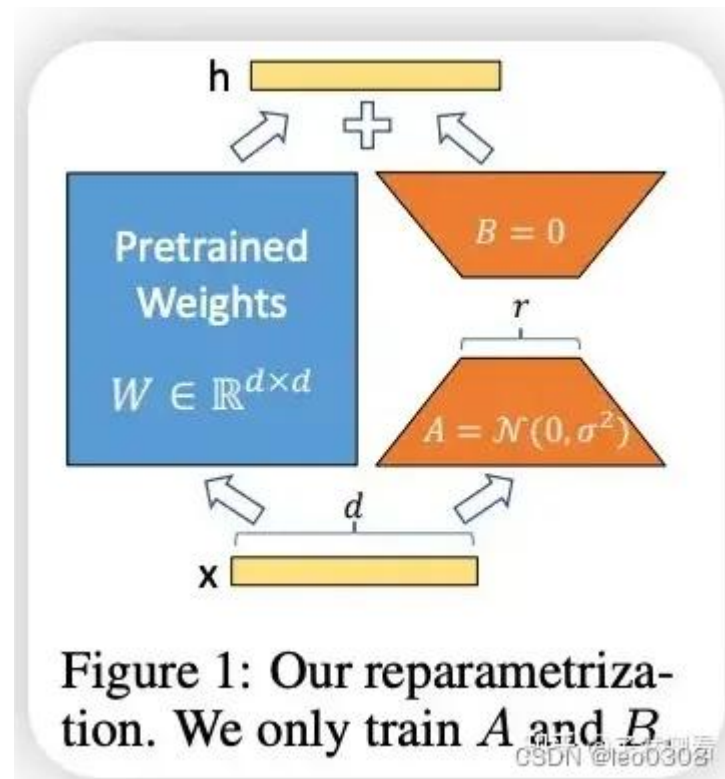
- 冻结原始权重 W ，不再更新。
- 新增两个低秩矩阵 A 和 B ，只训练它们。
- 更新量 = 输入先过 A ，再过 B ，最后缩放后加回 W 。

为什么有效

- **参数更少**：只更新小矩阵，训练成本下降 10-100 倍。
- **效果相近**：能接近全量微调的效果。
- **可插拔**：LoRA 适配器可独立保存、合并或替换。

工作流程

- 输入 $x \rightarrow$ 基础路径： $x \times W$
- 输入 $x \rightarrow$ LoRA 路径： $x \times A \rightarrow \times B \rightarrow$ 缩放
- 两条路径相加，得到最终输出 y



LoRA方法示意图

LORA微调参数的选择

- 在 Transformer 里，大部分可学习参数集中在 **线性变换矩阵**（如 Attention 的 W_q, W_v ，MLP 的投影矩阵）。
- 假设原始权重矩阵 $W \in \mathbb{R}^{d \times d}$ ，微调时需要加一个小的更新量 ΔW 。
- 关键假设： ΔW **通常是低秩的**（即有效信息集中在少量方向）。
- 因此用两个小矩阵近似： $\Delta W \approx AB, A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}, r \ll d$
- 这样只需要训练 $d \times r + r \times d = 2dr$ 个参数，而不是 d^2 。
- **传统微调**: 直接训练 **全部模型参数**（如 LLaMA 7B 有 70 亿参数），成本高 + 易过拟合。
- **LoRA 微调**: 只在某些层（如 Attention 的 Q/K/V 或 FFN 层）旁边加入 **两个小矩阵 A 和 B**：

1. 自注意力模块（Self-Attention）中最核心的 4 个矩阵

👉 LoRA 最常用于 Q 和 V（或全部 Q/K/V/O）

层名称	HuggingFace 常见参数名	用途
Query 变换矩阵	q_proj 或 query	生成 Query 向量
Key 变换矩阵	k_proj 或 key	生成 Key 向量
Value 变换矩阵	v_proj 或 value	生成 Value 向量
输出线性变换矩阵	o_proj 或 dense	多头 attention 输出整合

2. 前馈网络（Feed-Forward / MLP）中的线性层

👉 如果设置 --target_modules all-linear，也会对这些层使用 LoRA。

层名称	常见参数名	用途
第一层升维	fc1 / dense_in	扩展隐层维度
第二层降维	fc2 / dense_out	恢复维度

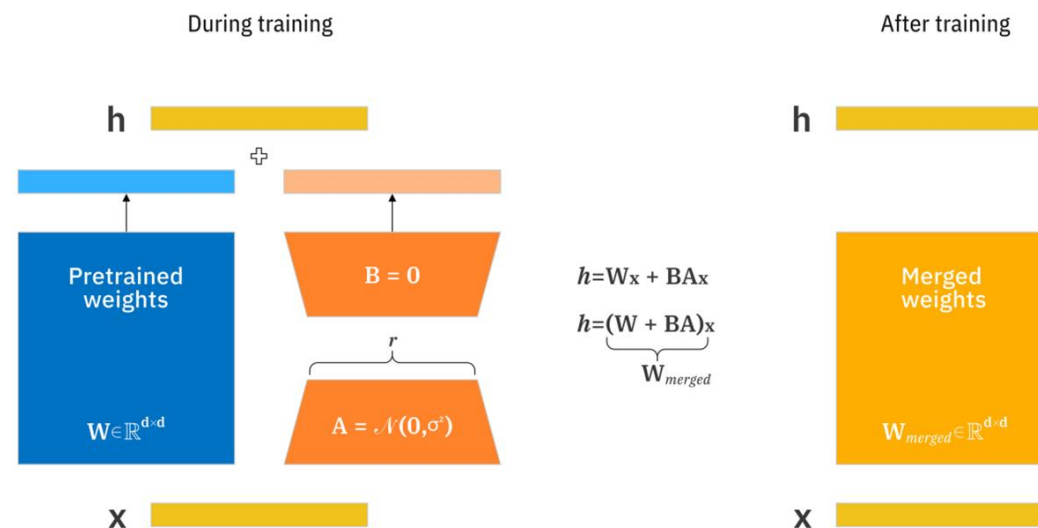
LoRA的数学表示

- 对于原始模型权重矩阵 $W \in \mathbb{R}^{d \times k}$, LoRA 用两个小矩阵 $A \in \mathbb{R}^{d \times r}$ 、 $B \in \mathbb{R}^{r \times k}$ 来表示其增量, 其中 $r \ll \min(d, k)$:

- $W' = W + \Delta W, \Delta W = \frac{\alpha}{r} \cdot A \cdot B$

- 其中:

- W 原始预训练权重 (冻结)
- ΔW LoRA 学习得到的权重增量
- A 降维矩阵 (可训练)
- B 升维矩阵 (可训练)
- r LoRA rank (低秩维度)
- α 缩放因子 (scale)
- W' 推理时最终模型权重



(1) r (秩, rank)

- 决定 **新增参数量大小** (越大 $\rightarrow$ 参数多 $\rightarrow$ 表达能力强 $\rightarrow$ 显存占用也更大)。
- 一般经验：
 - $r=4\sim8$: 轻量级微调, 适合小数据/实验。
 - $r=16\sim32$: 中等规模, 常用在 7B~13B 模型。
 - $r=64+$: 更大模型或任务更复杂时使用。

(2) `lora_alpha` (缩放因子)

- 用来调节 LoRA 分支对原始矩阵的影响。
- 常见经验: `lora_alpha` $\approx 2\sim4 * r$ 。
- 例如: $r=8 \rightarrow \text{alpha}=16$, $r=16 \rightarrow \text{alpha}=32$ 。
- 太大可能导致梯度不稳定, 太小会影响学习能力

(3) `lora_dropout`

- 防止过拟合。
- 小数据集推荐 $0.05 \sim 0.1$ 。
- 大数据集可以设为 0（几乎不需要 dropout）。

(4) `target_modules`

- 选择在哪些层注入 LoRA。
- 不同模型结构不同，最常见是：
 - `["q_proj", "v_proj"]` → 只在 Attention 的 Query/Value 投影矩阵上加 LoRA（推荐，计算量小，效果好）。
 - 有时也加 `["k_proj", "o_proj"]`，效果更好但显存更高。
 - 如果是 Qwen/LLaMA，默认用 Q/V；如果是 BERT 类，可能需要在 dense 层。

需求场景	推荐设置
快速实验 / 显存紧张	$r=4, \alpha=8, \text{dropout}=0.1$, 只对 Q/V 加 LoRA
常规模调 / 7B~13B 模型	$r=8\sim16, \alpha=16\sim32$, $\text{dropout}=0.05$
大规模任务 / 数据量多	$r=32+$, $\alpha=64+$, 可以加更多层 (Q/K/V/O)

数据条数	平均大小	典型应用
1k–5k	1–10 MB	跑通流程、实验教学
1万–5万	20–100 MB	小规模指令微调（LoRA）
10万–20万	200 MB–1 GB	行业/领域微调（医疗、金融）
100万+	5–10 GB	大规模全参数微调，研究机构/公司级

模型规模	方法	数据量	硬件	时间
1B - 3B	LoRA	1 - 5万	单卡 RTX 3090	2 - 4 小时
7B	LoRA	5 - 10万	单卡 A100-40G	6 - 12 小时
7B	LoRA	20万	2 × A100-80G	1 - 2 天
7B	全参数	10万+	8 × A100-80G	2 - 3 天
13B	全参数	50万+	16 × A100-80G	1 - 2 周
70B	全参数	100万+	上千 GPU	数周甚至更久

- **单轮任务型（翻译、摘要、分类、信息抽取）**

用 **instruction/input/output 格式** 就足够，写法简单直观。

```
{"instruction": "翻译成英文",  
  "input": "我爱北京天安门",  
  "output": "I love Tiananmen in Beijing."}
```

- **多轮对话 / 助手型任务（Chatbot, 问答系统）**

用 **ChatML 格式**，因为它有角色分隔，更贴近推理时的输入。

```
{"messages": [  
  {"role": "system", "content": "你是一个乐于助人的助手。"},  
  {"role": "user", "content": "我头痛两天，还有点恶心"},  
  {"role": "assistant", "content": "建议注意休息，多喝水，如症状加重请去医院检查。"}  
]}
```


主流微调工具对比



智泊AI
ZHIPU AI



魔泊云
MoPaaS

MS-Swift (ModelScope-Swift)

阿里巴巴通义团队

CLI一条命令微调

对国产模型（Qwen系列、ChatGLM等）支持最完善，内置Web UI界面，文档以中文为主，国内企业和高校广泛使用

适合：不想写代码、快速出结果；与Qwen系列搭配最佳

LLaMA-Factory

GitHub 40k+ stars, 国际最热门

零代码微调

完整Web界面（数据集选择→模型选择→超参数配置→训练监控），支持模型范围最广，国际社区活跃

适合：快速对比不同模型在同一任务上的效果

Unsloth

个人开发者团队，专注效率

速度快2~5倍 | 显存降60~80%

手写CUDA kernel、优化内存管理等底层技巧，速度比标准PEFT快2~5倍，显存降低60%~80%，同样硬件可训练更大模型

适合：个人开发者、小团队、单张消费级显卡、显存紧张

SWIFT微调框架

SWIFT微调框架 (ms-swift)



智泊AI
ZHIPO AI



魔泊云
MoPaaS

简介:

- 由 阿里达摩院 ModelScope 团队开发的轻量级大模型微调框架，专注于 快速上手、低门槛、高性能 的训练方式。
- 它基于 HuggingFace 生态，支持 LoRA / QLoRA / 全参数微调，提供命令行与 Web UI 两种方式。
- <https://github.com/modelscope/ms-swift>

Swift 的特点

- **简单易用：** 一条命令即可完成微调：
`swift sft --model Qwen/... --train_type lora --dataset ...`
- **无需自己写 Trainer：** 所有逻辑自动完成，包括数据加载、格式转换、训练与模型保存。
- **支持可视化 Web UI：** 可通过网页配置模型、数据和参数（适合教学和工程快速实验）。
- **适配主流模型：** 支持 Qwen、ChatGLM、LLaMA、Baichuan、DeepSeek 等。
- **无需大量显存：** 原生支持 LoRA / QLoRA，可在单张消费级 GPU 上训练

使用 Swift 命令行 (CLI)

```
CUDA_VISIBLE_DEVICES=0 swift sft \  
  --model Qwen/Qwen2.5-7B-Instruct \  
  --train_type lora \  
  --dataset 'AI-ModelScope/alpaca-gpt4-data-zh#500' \  
  --torch_dtype bfloat16 \  
  --num_train_epochs 1 \  
  --per_device_train_batch_size 1 \  
  --gradient_accumulation_steps 16 \  
  --learning_rate 5e-5 \  
  --lora_rank 8 \  
  --lora_alpha 16 \  
  --target_modules q_proj,v_proj \  
  --max_length 2048 \  
  --system '你是一个专业的AI助手，请严谨作答。' \  
  --output_dir output \  
  --seed 42
```

参数	含义
--model	基座模型
--train_type lora	LoRA 微调
--dataset	训练数据
--num_train_epochs	训练轮数
--learning_rate	学习率
--lora_rank / --lora_alpha	LoRA 参数
--gradient_accumulation_steps	累积梯度
--max_length	训练 token 上限
--output_dir	输出目录
--system	system prompt
--seed	随机数控制实验复现

什么是 Web UI?

Swift 提供了 **图形化训练界面 (Web UI)**，可以在网页中完成模型选择、数据配置、参数设置和训练启动，无需命令行或写代码，非常适合教学和快速实验。

如何启动 Web UI

```
pip install -U ms-swift  
SWIFT_UI_LANG=zh swift web-ui
```

执行后浏览器中打开 (默认端口 7860) :

 <http://127.0.0.1:7860/>

如需共享外网访问 (如 Colab 或服务器) :

```
SWIFT_UI_LANG=zh swift web-ui --share
```

Web UI 能做什么?

- 选择模型 (在线或本地)
- 选择训练数据集或上传本地 JSONL
- 配置训练方式 (LoRA / SFT / QLoRA / DPO 等)
- 设置超参数 (学习率、batch size、epoch)
- 一键启动微调
- 实时查看训练日志
- 查看训练记录与模型版本
- 导出模型用于部署 / 推理

使用 Python 进行 ms-swift 模型训练



智泊AI
ZHIPU AI



魔泊云
MoPaaS

```
from swift.llm import get_model_tokenizer, get_template
from swift.tuners import LoraConfig, Swift
from swift.llm import EncodePreprocessor, Seq2SeqTrainer
from datasets import load_dataset
from transformers import TrainingArguments

# 1. 加载模型 & Tokenizer
model, tokenizer = get_model_tokenizer(model_id)

# 2. 加载 Prompt 模板
template = get_template(template_type, tokenizer)

# 3. 添加 LoRA 模块
model = Swift.prepare_model(model, LoraConfig(r=8, lora_alpha=32))

# 4. 加载数据集并编码
dataset = load_dataset("json", data_files="data_clean.jsonl")
dataset = dataset.map(lambda x: EncodePreprocessor(template)(x))

# 5. 配置训练器并训练
trainer = Seq2SeqTrainer(model=model, args=TrainingArguments(...))
trainer.train()
```

模型训练超参数选择

这些超参数决定了一次训练中输入给模型的数据形态：

- **max length**
 - 单条输入序列的最大长度（token 数）。太长会截断，太短会 padding。
 - 影响显存和训练速度。
 - 小模型 / 短文本任务（分类、QA）：128 ~ 512；
 - 中长文本（指令微调、对话）：512 ~ 2048
- **per device train batch size**
 - 每张 GPU 上一次送入模型的样本数量。
 - 显存小 → batch size 小（1~2），显存大 → 可以更大。
 - 4090 / A100（24GB~80GB）：2 ~ 16
 - 普通消费级 GPU（8GB~12GB）：1 ~ 2
- **gradient accumulation_steps**
 - 梯度累积次数。
 - 比如 batch=2, steps=8 → 等效 batch=16。
 - 用来解决显存不足的问题。
- **eval batch size**
 - 验证时的 batch 大小，一般和训练一致或更大。

- 这些参数决定了 **模型如何学习**：
- **learning_rate**
 - 学习率，控制参数更新的步伐。太小 → 学得慢；太大 → loss 震荡或发散。
 - LoRA 通常用 $1e-4 \sim 2e-4$ 。
- **num_train_epochs**
 - 训练轮数。一个 epoch = 数据完整过一遍。
 - 数据多时通常只需要 1~3 轮。
- **warmup_steps / warmup_ratio**
 - 前几个 step 使用较小的学习率，再逐渐增大。防止训练初期不稳定。
- **weight_decay**
 - 权重衰减，避免参数过大导致过拟合。常用值：0.01。
- **optimizer**（通常用 AdamW）
 - 带有一阶/二阶矩估计，比 SGD 稳定。

这些参数决定了 **训练过程的稳健性**:

- **dropout / lora dropout**
 - 在训练中随机丢弃部分输入，避免过拟合。
 - LoRA 里一般设 0.05 ~ 0.1。
- **fp16 / bf16**
 - 混合精度训练，减少显存占用。
 - bf16 更稳定，适合 A100/H100；fp16 适合 V100。
- **gradient clipping (max_grad_norm)**
 - 限制梯度大小，避免梯度爆炸。
 - 常用值：1.0。
- **save strategy / save steps**
 - 定期保存模型（adapter 权重）。
- **evaluation strategy / eval steps**
 - 定期在验证集上计算 loss，防止过拟合。

实践：微调原理和实践

- 实践1. 深度学习练习
- 实践2. LoRA 原理示例
- 实践3. Swift 命令行练习
- 实践4. Swift 可视化界面练习
- 实践5. 微调程序及训练过程监测

- 详细内容参考练习手册